

Análise Comparativa de Desempenho e Escalabilidade: Abordagens Sequenciais e Paralelas na Extração de Dados via PokéAPI

Raphaela Samille Ramalho de Oliveira¹

¹Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco (IFPE)
Campus Belo Jardim – PE – Brasil

rsro@discente.ifpe.edu.br

Abstract. *This report investigates the performance of different concurrency models in Python for retrieving and storing images from the PokéAPI. The study compares a sequential baseline against threading, multiprocessing, and concurrent.futures architectures. By analyzing execution times under various workloads (100, 500, and 1,000 images) and worker counts (2, 4, and 8), we quantify the impact of parallelism on I/O-bound systems. Results indicate that thread-based pooling provides the most consistent scalability, significantly reducing latency.*

Resumo. *Este relatório investiga o desempenho de diferentes modelos de concorrência em Python para a recuperação e armazenamento de imagens da PokéAPI. O estudo compara uma linha de base sequencial contra arquiteturas de threading, multiprocessing e concurrent.futures. Ao analisar os tempos de execução sob várias cargas de trabalho (100, 500 e 1.000 imagens) e contagens de workers (2, 4 e 8), quantificamos o impacto do paralelismo em sistemas I/O-bound. Os resultados indicam que o pool baseado em threads oferece a escalabilidade mais consistente, reduzindo significativamente a latência.*

1. Contexto e Objetivos

A coleta automatizada de dados por meio de APIs RESTful é uma tarefa comum na Engenharia de Software, mas costuma esbarrar no gargalo da latência de rede. A PokéAPI [1], que disponibiliza dados sobre Pokémon, foi utilizada neste projeto como ambiente de teste para operações do tipo **I/O-bound**. O objetivo principal é baixar as imagens (*sprites*) frontais de subconjuntos de Pokémons e salvá-las em disco.

O foco deste trabalho é comparar uma implementação sequencial padrão com três abordagens paralelas disponíveis no Python: *threading*, *multiprocessing* e *concurrent.futures*. A análise busca medir como a divisão de tarefas entre múltiplas unidades de execução (*workers*) otimiza o tempo total de download, aproveitando o tempo em que o processador ficaria ocioso esperando as respostas do servidor.

2. Metodologia e Implementação

Os testes foram realizados em um notebook Samsung Galaxy Book 4, equipado com processador Intel de 10 núcleos físicos e 12 lógicos, rodando o sistema operacional Windows 11. O sistema foi desenvolvido em Python 3.14, utilizando a biblioteca *requests* para os testes HTTP e a biblioteca nativa *os* para a gravação dos arquivos no disco.

Os cenários de testes e as diretrizes de avaliação de desempenho seguiram estritamente o roteiro prático proposto para a atividade [3], mapeando o comportamento dos modelos conforme documentado no repositório base da disciplina [2]. Para avaliar o comportamento dos modelos, os scripts foram estruturados para baixar volumes de 100, 500 e 1.000 imagens. Nas abordagens paralelas, foram testados cenários com 2, 4 e 8 *workers*. Como a conexão de rede pode oscilar, cada configuração foi executada 10 vezes consecutivas para extrair uma média confiável dos tempos de execução. Além disso, o algoritmo limpava a pasta de destino a cada ciclo (usando o módulo *shutil*) para evitar que o cache do sistema operacional mascarasse os resultados de escrita.

3. Resultados Obtidos

Os tempos médios cronometrados foram organizados na Tabela 1, permitindo comparar diretamente o desempenho de cada método de acordo com o aumento do número de *workers*.

Tabela 1. Tempos Médios de Execução (Média de 10 repetições por cenário)

Método	Workers	100 Imagens	500 Imagens	1.000 Imagens
<i>Sequential</i>	1	63.32s	1272.96s	1431.27s
<i>Threads</i>	2	35.28s	415.97s	580.67s
<i>Multiprocessing</i>	2	43.04s	235.49s	603.31s
<i>Futures</i>	2	30.81s	426.46s	638.34s
<i>Threads</i>	4	26.43s	230.55s	478.88s
<i>Multiprocessing</i>	4	38.51s	252.33s	423.69s
<i>Futures</i>	4	27.77s	235.19s	507.43s
<i>Threads</i>	8	22.22s	157.81s	981.64s
<i>Multiprocessing</i>	8	45.23s	146.10s	3596.37s
<i>Futures</i>	8	24.69s	205.84s	405.40s

Para facilitar a análise visual do comportamento do sistema com o aumento da carga de trabalho, os tempos médios foram consolidados nos gráficos a seguir:

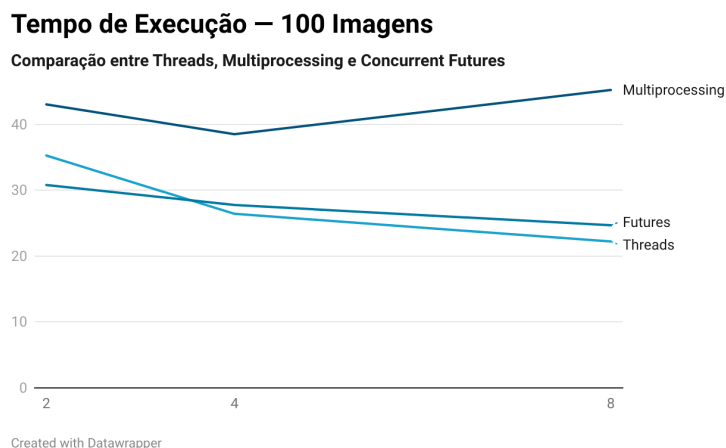


Figura 1. Curva de desempenho para a carga de 100 imagens.

Tempo de Execução — 500 Imagens

Análise de desempenho com diferentes quantidades de workers

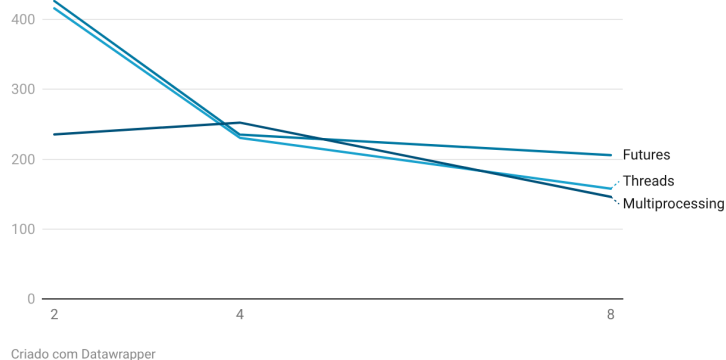


Figura 2. Curva de desempenho para a carga de 500 imagens.

Tempo de Execução — 1.000 Imagens

Escalabilidade e estabilidade sob alta carga

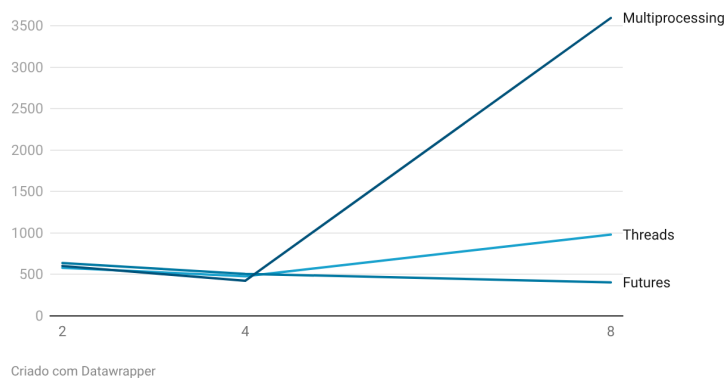


Figura 3. Comportamento sob estresse de carga de 1.000 imagens.

4. Discussão Técnico-Experimental

Os resultados mostram que a concorrência reduziu drasticamente o tempo de execução em comparação ao modelo sequencial tradicional. No entanto, cada método respondeu de forma diferente dependendo da quantidade de requisições.

Nas cargas de 100 e 500 imagens, aumentar o número de *workers* trouxe melhorias constantes. O *Multiprocessing* e as *Threads* atingiram suas maiores velocidades ao utilizar 8 *workers*, mostrando que o processamento paralelo funcionou bem nesses cenários intermediários.

O cenário mudou completamente no teste de 1.000 imagens com 8 *workers*. Houve uma perda severa de eficiência no *Multiprocessing*, cujo tempo disparou para 3596.37 segundos (quase uma hora), e nas *Threads*, que subiram para 981.64 segundos.

Esse problema acontece por dois motivos principais. Primeiro, o sistema operacional Windows gasta muita energia para criar, alternar e fechar processos pesados (*overhead*). Segundo, o envio de muitas requisições simultâneas por vários processos acabou ativando defesas de segurança na PokéAPI (como o bloqueio temporário ou *rate limiting*), travando as respostas do servidor. O módulo *Concurrent Futures* foi o mais

estável e seguro nessa situação de estresse máximo, segurando o tempo em 405.40s graças ao seu gerenciador otimizado de fila.

5. Análise de Gargalos e Relato da Experiência

Concluir toda essa bateria de testes exigiu um effort longo do computador e da rede. Ao somar todas as variações necessárias (3 volumes de imagens $\times$ 3 contagens de *workers* $\times$ 10 repetições cada, além do controle sequencial), o programa disparou milhares de conexões HTTP, resultando em quase 5 horas de execução contínua.

Durante esse processo, ficou claro que o desempenho foi limitado por dois grandes fatores:

- **Gargalo na Rede Externa:** Como o programa passa a maior parte do tempo esperando arquivos baixarem (I/O-bound), a velocidade final ficou muito dependente da estabilidade da PokéAPI. No volume de 1.000 imagens, o servidor remoto começou a oscilar e demorar mais para responder quando atacado por 8 *workers* ao mesmo tempo. Isso prova que o limite do desempenho não era a falta de processador no notebook, mas sim a capacidade da rede em trazer os dados.
- **Custo de Gerenciamento do Sistema Operacional:** O monitoramento do Galaxy Book 4 mostrou que o uso de CPU ficou baixo para *Threads* e *Futures*, o que é normal enquanto o sistema espera a rede. Mas no *Multiprocessing* com 8 *workers*, abrir tantas instâncias do Python consumiu muita memória e gerou disputa na hora de salvar os arquivos no disco. No Windows, o custo de gerenciar esses processos pesados superou a vantagem de paralelizar a tarefa, causando o pico de tempo observado no gráfico.

Essa experiência prática mostra que, ao planejar sistemas concorrentes, não basta olhar apenas para o código. É preciso prever o comportamento das APIs externas e as limitações do sistema operacional onde o software vai rodar.

6. Conclusão

O projeto comprovou que o uso de programação paralela é fundamental para tarefas que dependem de operações de rede (I/O-bound). O notebook utilizado lidou muito bem com o escalonamento até 4 *workers*, mas apresentou limitações ao subir para 8 na carga máxima devido a bloqueios na API externa e custos do sistema operacional. Com base nos testes, conclui-se que o *Concurrent Futures* entrega o melhor equilíbrio entre facilidade de código e estabilidade para coletas de dados em larga escala.

Referências

- [1] PokéAPI. (2026). *The RESTful Pokémon API*. Disponível em: <https://pokeapi.com/>. Acesso em: 19 mai. 2026.
- [2] Souza, L. (2026). *Parallel and Distributed Systems*. Repositório GitHub. Disponível em: <https://github.com/levysouza/Parallel-and-Distributed-Systems>. Acesso em: 19 mai. 2026.
- [3] Souza, L. (2026). *Atividade Prática: Análise de Concorrência e Sistemas Computacionais I/O-Bound*. Google Classroom — IFPE Campus Belo Jardim. Disponível em: <https://classroom.google.com/c/Nzk3MTI1OTI0NzI2/p/Nzk3NDkyNDk1MTcy/details>. Acesso em: 19 mai. 2026.